

IoT Ergonomic Posture Monitor

Aaryans Nepal

CSC 494 — IoT (Spring 2026) — Final Presentation



BAD POSTURE (EXAMPLES OF)

Painful Symptoms from Bad Posture

- Upper or lower back pain
- Neck, shoulder and arm pain
- Lower limb pain, including pain in the hip, knee or ankle
- Muscle fatigue
- Headaches caused by a build-up of tension in the upper back, neck and shoulders



Health Risk

Incontinence

- Slouching increases pressure on the bladder
- Weakens pelvic floor support → urine leakage (e.g., laughing/coughing)

Constipation

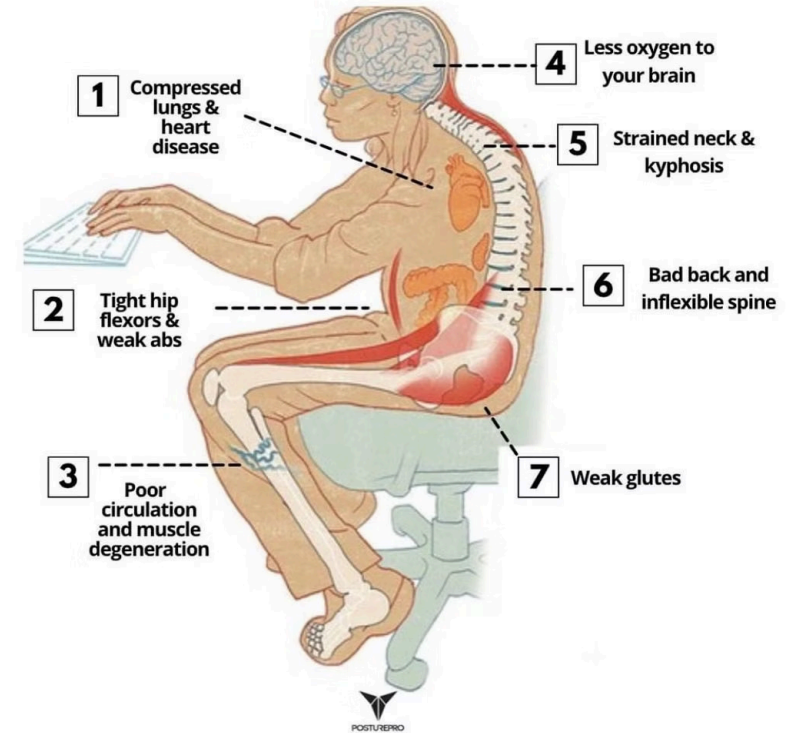
- Poor toilet posture (knees below hips) makes bowel movements harder
- Can lead to straining, hard stools, and incomplete evacuation

Heartburn & Slow Digestion

- Slouching after eating can cause acid reflux (heartburn)
- May slow down digestion in the intestines

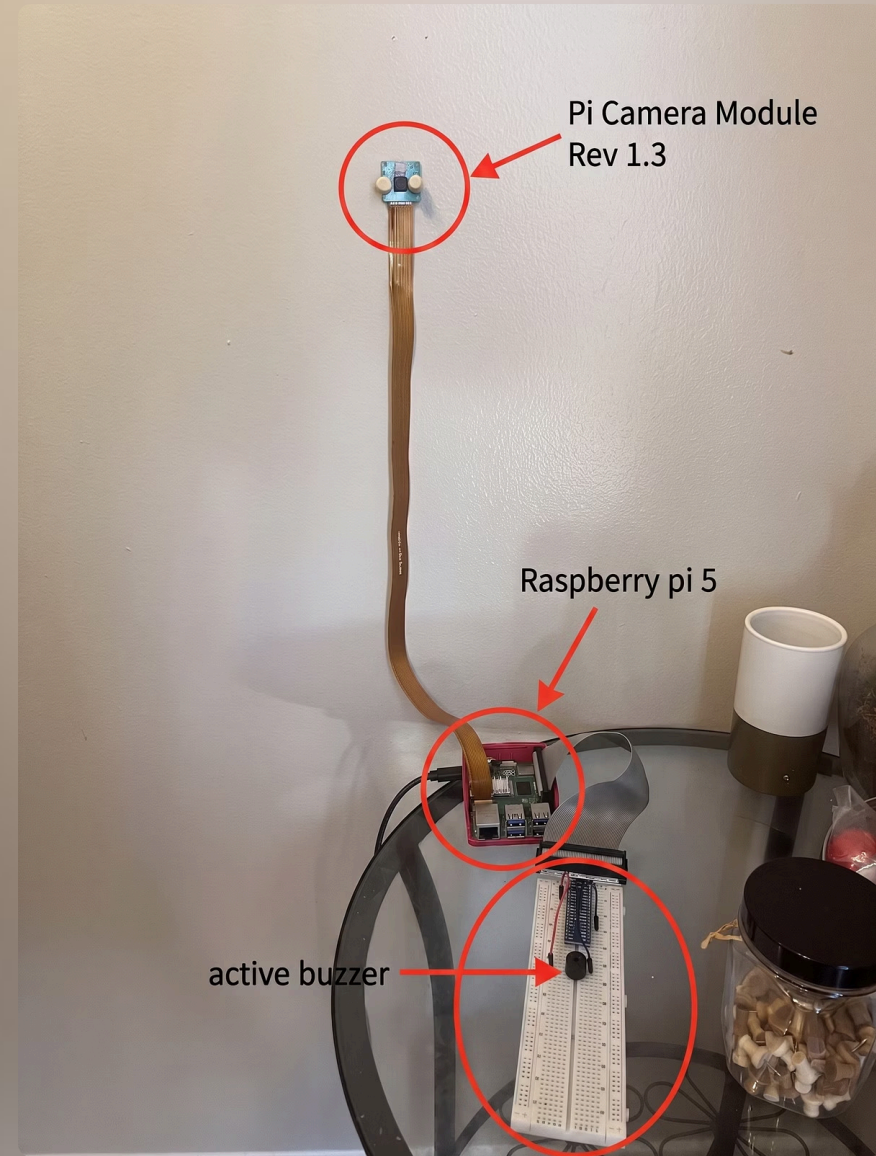
Sources: <https://www.health.harvard.edu/healthy-aging-and-longevity/3-surprising-risks-of-poor-posture>

EFFECTS OF SITTING POSTURE



What I Built

- Pi Camera watches me at my desk
- Live feed streams to the Raspberry Pi
- Pi tracks my pose and scores it **0–100**
- Slouch → **buzzer on**
- Sit up → **buzzer off**



Tech Stack

Hardware

- Raspberry Pi 5 (4 GB RAM)
- Pi Camera Rev 1.3
- Active buzzer on GPIO 17

Software

- Python 3 on Raspberry Pi OS
- MediaPipe Pose (BlazePose) — pose estimation, 8-12 FPS on CPU
- OpenCV — video capture + frame processing
- picamera2 — live Pi Camera driver
- Flask + MJPEG — live browser stream
- gpiozero — buzzer control
- CSV + JSON — per-session data logging
- ZeroTier — remote SSH from anywhere

Why this stack

- Runs on CPU only — no GPU, no cloud
- Private — no data ever leaves the Pi
- Cheap — total hardware under \$100
- Standard tools — easy to fork, easy to teach

How It Works: Input & Processing

01

Frame Capture

- Every frame is read at roughly 10 times per second.
- **picamera2 library** (640×480 RGB888)
- Fallback to **cv2.VideoCapture(0)** OpenCV for .MOV/.mp4

Two versions of every frame:

1. **frame_bgr** — the BGR version (what OpenCV expects for drawing overlays + JPEG encoding)
 2. **frame_rgb** — the RGB version (what MediaPipe requires as input)
- OpenCV handles the color swap with **cv2.cvtColor(frame, COLOR_BGR2RGB)**.

03

Key Point Extraction + Visibility Gating

- Extract 7 key points:
 - Nose
 - Left ear, Right ear
 - Left eye, Right eye
 - Left shoulder, Right shoulder
- Visibility threshold: < 0.5 = frame rejected

02

Pose Estimation (MediaPipe BlazePose)

- **frame_rgb** is fed into **mediapipe.solutions.pose.Pose** — Google's BlazePose model.
- Two-stage neural network: Detector + Landmark tracker
- Detects person, then tracks 33 body keypoints
- FPS jumps to 8-12 after first frame (no re-detection)
- Each landmark: x, y, z (depth), visibility score
- BlazePose runs on **CPU** using TensorFlow Lite with the **XNNPACK** delegate.

04

The 5 Metrics

- **nose_y**: vertical position
- **nose_z_depth**: forward lean
- **ear_z_depth**: head posture
- **face_size**: distance nose to ear
- **shoulder_tilt**: sideways lean

How It Works: Analysis & Output

01

Scoring

- Self-calibrate mode: 5-second baseline, then drift detection
- Weights: nose_y (30%), nose_z_depth (25%), ear_z_depth (20%), face_size (15%), shoulder_tilt (10%) - pulled from diagnostic data
- Research mode: absolute thresholds from ergonomics research
- Output: raw score 0-100 per frame

03

Buzzer

- 10 seconds continuous BAD → BUZZER ON
- 3 seconds continuous GOOD → BUZZER OFF
- GPIO 17 driven by gpiozero library

02

Smoothing + Hysteresis

- EMA smoothing ($\alpha = 0.2$) removes jitter from inferred Z-axis
- Hysteresis: flip GOOD→BAD at 45, BAD→GOOD at 35
- Prevents threshold thrashing

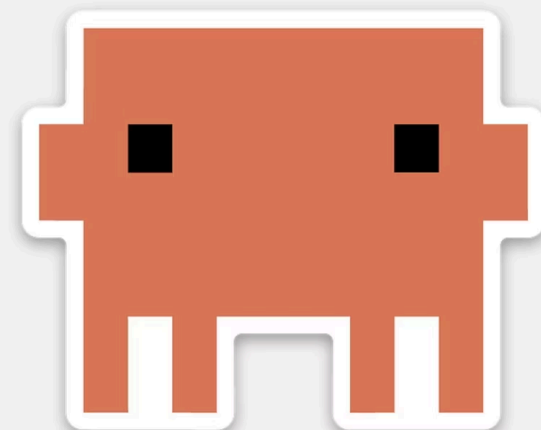
04

Web Stream & Data Logging (Parallel)

- Background thread runs pose estimation
- Flask web server on port 5000 serves MJPEG stream
- Every frame logged to CSV: metrics + score + state
- Line-buffered for data integrity

Learning with AI

- **Claude, ChatGPT, Copilot** — great coding partners.
- **AI models are great "yes men."** They agree with your hypothesis, make you feel like a genius, and always produce a paper or snippet that fits your story.
- One prompt — *"this feels like overkill with no real endpoint"* — and the model completely flipped, happily listing every flaw the project had. The flaws hadn't changed. My framing had.
- **The rule I keep:** AI confirms the hypothesis you give it. Your job is to challenge it with data.



Issues Encountered and How I Solved Them

- **X/Y metrics stuck at 0-2 / 100** → wrote `diagnose.py` to dump every landmark delta between good/bad frames. Rebuilt around Z-axis.
- **Pi Camera broken** → spent weeks assuming cables; Prof. Ross helped isolate the camera itself. Swapped to Rev 1.3.
- **Calibration failing silently** → MediaPipe was returning 0 valid samples because no one was in frame during the 5-second window. Added clearer prompts and terminal guidance.
- **Pivot** → Started with theft detection but changed it to posture monitoring



Where this Goes — Sports Coaching, Gym / Yoga

- The system I built is **general**. Swap the metrics and thresholds, and the same pipeline works for any movement that a coach cares about.
- The future version I want to build: a **basketball** (or any suitable sport) variant that sits courtside during practice and gives coaches an objective, per-player read on form — shooting stance, elbow alignment, follow-through, foot positioning.



Links

- GitHub (project): https://github.com/AaryansNepal/CSC-494_Project
- GitHub (Learning with AI): https://github.com/AaryansNepal/CSC-494_Learning-with-AI
- **Demo video:** see `demo/` folder in the project repo

Thank You

Questions?

— Aaryans Nepal, CSC 494 — IoT, Spring 2026, Northern Kentucky University